Listas en Java

Ángel de Jesús Nolasco Rivera
Grupo: GTID244
Estructuras de datos
Fecha: 23/10/2026
Repositorio: https://github.com/ANGEL7231/angel-nolasco-listas-java

Ejercicio 2. cuenta_letras_al_vector

Descripción del problema

El programa recibe una palabra escrita con letras minúsculas, de hasta 1000 caracteres, que también puede venir vacía. Hay que contar cuántas veces aparece cada letra del abecedario. Primero se imprime una línea con las 26 cantidades y después una línea por cada letra que sí apareció, en orden alfabético.

Tipo de lista y por qué la elegí

Usé una lista simple de 26 nodos, uno para cada letra. Siempre se recorre de la a a la z, así que no necesito ir hacia atrás y la lista doble no me aporta nada. Cada nodo guarda su letra y un contador, y como las letras ya están en orden, imprimir en orden alfabético es solo recorrer la lista.

Explicación del algoritmo

Primero creo los 26 nodos con el contador en cero. Después, por cada carácter de la palabra recorro la lista con actual = actual.siguiente hasta encontrar el nodo de esa letra y le sumo uno. Al final recorro la lista para imprimir todos los contadores en la primera línea y la recorro otra vez para escribir solo las letras que tienen contador mayor que cero.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

Caso 1: Normal (enunciado)

Entrada:

```
programacion
```

Salida esperada:

```
2 0 1 0 0 0 1 0 1 0 0 0 1 1 2 1 0 2 0 0 0 0 0 0 0 0
La letra a aparece 2 veces
La letra c aparece 1 veces
La letra g aparece 1 veces
La letra i aparece 1 veces
La letra m aparece 1 veces
La letra n aparece 1 veces
La letra o aparece 2 veces
La letra p aparece 1 veces
La letra r aparece 2 veces
```

```
=== Caso 1: normal ===
Entrada:
programacion
Salida:
2 0 1 0 0 0 1 0 1 0 0 0 1 1 2 1 0 2 0 0 0 0 0 0 0 0
La letra a aparece 2 veces
La letra c aparece 1 veces
La letra g aparece 1 veces
La letra i aparece 1 veces
La letra m aparece 1 veces
La letra n aparece 1 veces
La letra o aparece 2 veces
La letra p aparece 1 veces
La letra r aparece 2 veces
```

Caso 2: Con repeticiones (enunciado)

Entrada:

```
aaaaaccccceeegghxxzzz
```

Salida esperada:

```
5 0 4 0 3 0 2 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 3
La letra a aparece 5 veces
La letra c aparece 4 veces
La letra e aparece 3 veces
La letra g aparece 2 veces
La letra h aparece 1 veces
La letra x aparece 2 veces
La letra z aparece 3 veces
```

```
=== Caso 2: con repeticiones ===
Entrada:
aaaaaccccceeegghxxzzz
Salida:
5 0 4 0 3 0 2 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 3
La letra a aparece 5 veces
La letra c aparece 4 veces
La letra e aparece 3 veces
La letra g aparece 2 veces
La letra h aparece 1 veces
La letra x aparece 2 veces
La letra z aparece 3 veces
```

Caso 3: Límite (cadena vacía)

Entrada:

```
(cadena vacía)
```

Salida esperada:

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
=== Caso 3: limite (cadena vacia) ===
Entrada:
(cadena vacía)
Salida:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Caso 4: Límite (un carácter)

Entrada:

```
z
```

Salida esperada:

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
La letra z aparece 1 veces
```

```
=== Caso 4: limite (un caracter) ===
Entrada:
z
Salida:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
La letra z aparece 1 veces
```

Caso 5: Grande (1000 caracteres)

Entrada:

```
El abecedario repetido hasta completar 1000 caracteres
```

Salida esperada:

```
39 39 39 39 39 39 39 39 39 39 39 38 38 38 38 38 38 38 38 38 38 38 38
38
(26 líneas, una por cada letra)
```

```
=== Caso 5: grande (1000 caracteres) ===
Primera línea de la salida:
39 39 39 39 39 39 39 39 39 39 39 38 38 38 38 38 38 38 38 38 38 38 38
 38
Letras que aparecen: 26 (esperado: 26)
```

Veredicto de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:00	java	100.00%	Terminada	Correcta	0.00 MB	0.14 s	

Enlace al código

[angel-nolasco-listas-java/ejercicio2-cuenta-letras-al-vector at main · ANGEL7231/angel-nolasco-listas-java]

Conclusiones

La dificultad principal fue el caso de la cadena vacía: readLine() puede devolver null o una línea en blanco y, si no se controla, el programa falla; lo resolví convirtiendo null en una cadena vacía. Otra dificultad fue el formato: los números van separados por espacios sin un espacio sobrante al final, y el detalle solo se imprime para las letras que sí aparecen.

Ejercicio 4. Inserción en Lista Enlazada

Descripción del problema

El programa lee una lista de números enteros positivos, uno por línea, hasta que se acaba la entrada. Hay que insertar un nodo con el valor -1 justo antes de cada nodo que tenga un número impar, contar cuántos nodos se insertaron e imprimir esa cantidad y después la lista con el formato 1 -> 2 -> NULL.

Tipo de lista y por qué la elegí

Usé una lista simple. Solo recorro la lista una vez de principio a fin y para insertar antes de un nodo me basta con ir guardando en una variable el nodo anterior, así que no necesito que cada nodo tenga su propio puntero anterior. También guardé el último nodo para cargar los datos más rápido.

Explicación del algoritmo

Recorro la lista con dos punteros, anterior (que empieza en null) y actual (que empieza en la cabeza). Si el valor de actual es impar creo un nodo con -1, hago que apunte a actual (nuevo.siguiente = actual) y luego hago que el anterior apunte al nuevo (anterior.siguiente = nuevo). Si anterior es null significa que el impar era la cabeza, entonces el nuevo nodo pasa a ser la cabeza. Después avanzo con anterior = actual y actual = actual.siguiente, siempre desde actual y no desde el nodo nuevo, para no revisar otra vez el -1 que acabo de insertar. Al final imprimo la cantidad insertada y recorro la lista para mostrarla.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

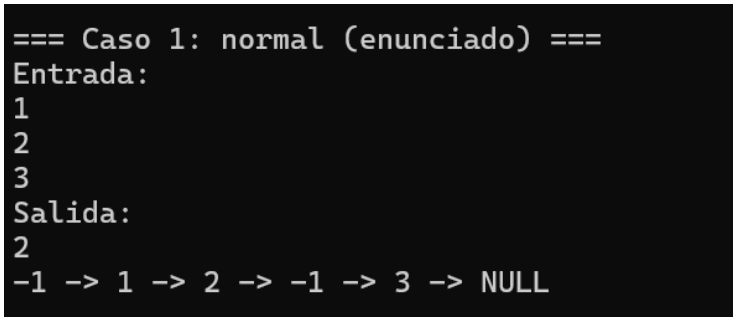
Caso 1: Normal (enunciado)

Entrada:

```
1
2
3
```

Salida esperada:

```
2
-1 -> 1 -> 2 -> -1 -> 3 -> NULL
```



Caso 2: Mezcla de pares e impares

Entrada:

```
4
5
8
3
3
10
```

Salida esperada:

```
3
4 -> -1 -> 5 -> 8 -> -1 -> 3 -> -1 -> 3 -> 10 -> NULL
```

```
=== Caso 2: mezcla ===
Entrada:
4
5
8
3
3
10
Salida:
3
4 -> -1 -> 5 -> 8 -> -1 -> 3 -> -1 -> 3 -> 10 -> NULL
```

Caso 3: Límite (lista vacía)

Entrada:

```
(sin valores)
```

Salida esperada:

```
0
NULL
```

```
=== Caso 3: limite (lista vacia) ===
Entrada:
(lista vacía)
Salida:
0
NULL
```

Caso 4: Límite (un solo impar)

Entrada:

```
7
```

Salida esperada:

```
1
-1 -> 7 -> NULL
```

```
=== Caso 4: limite (un solo impar) ===
Entrada:
7
Salida:
1
-1 -> 7 -> NULL
```

Caso 5: Todos pares

Entrada:

```
2
4
6
```

Salida esperada:

```
0
2 -> 4 -> 6 -> NULL
```

```
=== Caso 5: todos pares ===
Entrada:
2
4
6
Salida:
0
2 -> 4 -> 6 -> NULL
```

Caso 6: Todos impares

Entrada:

```
1
3
5
```

Salida esperada:

```
3
-1 -> 1 -> -1 -> 3 -> -1 -> 5 -> NULL
```

```
=== Caso 6: todos impares ===
Entrada:
1
3
5
Salida:
3
-1 -> 1 -> -1 -> 3 -> -1 -> 5 -> NULL
```

Caso 7: Grande (1 al 10000)

Entrada:

```
Valores del 1 al 10000, uno por línea
```

Salida esperada:

```
5000 nodos insertados
tamaño final de la lista: 15000
```

```
=== Caso 7: grande (N = 10000) ===
Nodos insertados: 5000 (esperado: 5000)
Tamaño final de la lista: 15000 (esperado: 15000)
Tiempo: 0 ms
```

Veredicto de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:16	java	100.00%	Terminada	Correcta	0.91 MB	0.66 s	

Tienes que esperar 34 segundos antes de que puedas hacer otro envío a este problema

Enlace al código

[angel-nolasco-listas-java/ejercicio4_InsercionEnListaEnlazada at main · ANGEL7231/angel-nolasco-listas-java]

Conclusiones

La dificultad principal fue el orden de los enlaces al insertar: si se cambia anterior.siguiente antes de hacer nuevo.siguiente = actual, se pierde el resto de la lista; lo resolví enlazando primero el nuevo nodo hacia adelante y después desde el anterior. Otra dificultad fue el caso en que el primer nodo es impar, porque no existe un anterior; lo resolví actualizando la cabeza.

Ejercicio 6. Cajas de dulces

Descripción del problema

En un estante hay N cajas de dulces en fila y Carlitos solo puede llevarse K cajas que estén una junto a la otra. El programa lee N, K y los dulces de cada caja y calcula la mayor cantidad de dulces que se puede llevar.

Tipo de lista y por qué la elegí

Usé una lista simple. Las cajas se recorren siempre de izquierda a derecha y solo necesito dos referencias que avanzan hacia adelante, una al inicio y otra al final del grupo de K cajas, así que una lista doble no me da nada extra.

Explicación del algoritmo

Primero sumo las primeras K cajas con un puntero fin, que queda en la K-ésima caja; esa es la primera suma y la guardo como máximo. Luego voy moviendo el grupo una caja a la vez mientras fin.siguiente no sea null: fin avanza y sumo su valor, y resto el valor de la caja de inicio, que también avanza. Si la suma nueva es mayor que el máximo, la guardo. Así cada caja entra y sale una sola vez en lugar de volver a sumar las K cajas cada vez.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

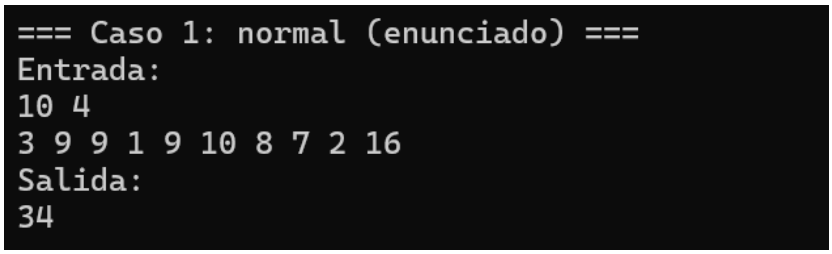
Caso 1: Normal (enunciado)

Entrada:

```
10 4
3 9 9 1 9 10 8 7 2 16
```

Salida esperada:

```
34
```



Caso 2: Normal (enunciado)

Entrada:

```
5 2
9 5 2 10 3
```

Salida esperada:

```
14
```

```
=== Caso 2: normal (enunciado) ===
Entrada:
5 2
9 5 2 10 3
Salida:
14
```

Caso 3: Límite (K = N)

Entrada:

```
5 5
1 2 3 4 5
```

Salida esperada:

```
15
```

```
=== Caso 3: limite (K = N) ===
Entrada:
5 5
1 2 3 4 5
Salida:
15
```

Caso 4: Límite (K = 1)

Entrada:

```
6 1
4 9 2 9 1 7
```

Salida esperada:

```
9
```

```
=== Caso 4: limite (K = 1) ===
Entrada:
6 1
4 9 2 9 1 7
Salida:
9
```

Caso 5: Límite (una sola caja)

Entrada:

```
1 1
500
```

Salida esperada:

```
500
```

```
=== Caso 5: limite (N = 1) ===
Entrada:
1 1
500
Salida:
500
```

Caso 6: Mejor ventana al inicio

Entrada:

```
5 3
8 9 7 1 1
```

Salida esperada:

```
24
```

```
=== Caso 6: mejor ventana al inicio ===
Entrada:
5 3
8 9 7 1 1
Salida:
24
```

Caso 7: Mejor ventana al final

Entrada:

```
6 3
1 1 1 1 9 9
```

Salida esperada:

```
19
```

```
=== Caso 7: mejor ventana al final ===
Entrada:
6 3
1 1 1 1 9 9
Salida:
19
```

Caso 8: Grande (N = 1000, K = 500)

Entrada:

```
1000 500
valores 1, 2, … 500 y luego 500, 499, … 1
```

Salida esperada:

```
187750
```

```
=== Caso 8: grande (N = 1000, K = 500) ===
Suma máxima: 187750 (esperado: 187750)
Tiempo: 0 ms
```

Veredicto de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:26	java	100.00%	Terminada	Correcta	0.00 MB	0.57 s	

Tienes que esperar 44 segundos antes de que puedas hacer otro envío a este problema

Enlace al código

[angel-nolasco-listas-java/ejercicio6_CajasDeDulces at main · ANGEL7231/angel-nolasco-listas-java]

Conclusiones

La dificultad principal fue entender que las cajas deben ser consecutivas: al principio pensé en elegir las K cajas con más dulces, y eso daría 51 en el primer ejemplo en lugar de 34. Otra dificultad fue colocar bien el puntero fin al terminar la primera suma: debe quedar en la K-ésima caja y no una después; lo resolví empezando en la cabeza y avanzando K-1 veces.

Ejercicio 8. Derdenio y sus Estructuras de Datos

Descripción del problema

Derdenio tiene dos listas de N números enteros, cada una ordenada de menor a mayor, y quiere juntarlas en una sola lista de 2N números ordenada de mayor a menor. El programa lee las dos listas e imprime la lista combinada.

Tipo de lista y por qué la elegí

Usé listas dobles. Como las listas vienen de menor a mayor y la salida debe ser de mayor a menor, me conviene empezar por el final de cada lista, donde está su número más grande, e ir retrocediendo con el puntero anterior. Con una lista simple tendría que invertir primero las listas, y con la doble las recorro al revés directamente.

Explicación del algoritmo

Cargo cada lista con agregarFinal y guardo la referencia a su último nodo. Pongo un puntero p1 en el último nodo de la primera lista y otro p2 en el último de la segunda. Mientras ninguno sea null comparo p1.valor con p2.valor, agrego el mayor al final de la lista resultado y retrocedo ese puntero con anterior. Cuando una lista se acaba paso lo que queda de la otra, que ya viene en orden. Al final imprimo la lista resultado de la cabeza a la cola.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

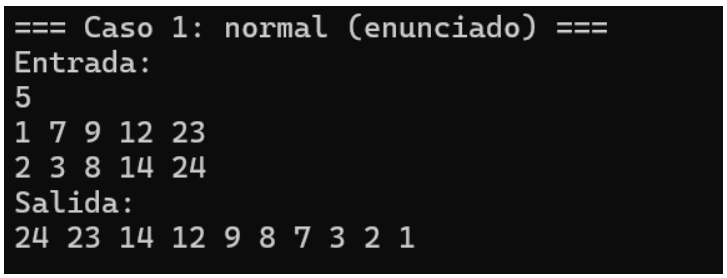
Caso 1: Normal (enunciado)

Entrada:

```
5
1 7 9 12 23
2 3 8 14 24
```

Salida esperada:

```
24 23 14 12 9 8 7 3 2 1
```



Caso 2: Límite (N = 1)

Entrada:

```
1
5
3
```

Salida esperada:

```
5 3
```

```
=== Caso 2: limite (N = 1) ===
Entrada:
1
5
3
Salida:
5 3
```

Caso 3: Números repetidos

Entrada:

```
3
1 2 2
2 2 3
```

Salida esperada:

```
3 2 2 2 2 1
```

```
=== Caso 3: repetidos ===
Entrada:
3
1 2 2
2 2 3
Salida:
3 2 2 2 2 1
```

Caso 4: Una lista mayor que la otra

Entrada:

```
3
1 2 3
10 20 30
```

Salida esperada:

```
30 20 10 3 2 1
```

```
=== Caso 4: una lista mayor que la otra ===
Entrada:
3
1 2 3
10 20 30
Salida:
30 20 10 3 2 1
```

Caso 5: Con negativos y cero

Entrada:

```
3
-5 -1 0
-4 2 6
```

Salida esperada:

```
6 2 0 -1 -4 -5
```

```
=== Caso 5: con negativos ===
Entrada:
3
-5 -1 0
-4 2 6
Salida:
6 2 0 -1 -4 -5
```

Caso 6: Grande (N = 5000)

Entrada:

```
5000
pares 0, 2, 4 … 9998
impares 1, 3, 5 … 9999
```

Salida esperada:

```
10000 números, de 9999 a 0 (de mayor a menor)
```

```
=== Caso 6: grande (N = 5000) ===
Cantidad de números: 10000 (esperado: 10000)
Primero: 9999 (esperado: 9999)
Último:  0 (esperado: 0)
Tiempo: 9 ms
```

Veredicto de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:36	java	100.00%	Terminada	Correcta	0.00 MB	0.70 s	

Tienes que esperar 43 segundos antes de que puedas hacer otro envío a este problema

Enlace al código

[angel-nolasco-listas-java/ejercicio8_Derdenio at main · ANGEL7231/angel-nolasco-listas-java]

Conclusiones

La dificultad principal fue entender de dónde empezar: al principio recorría las listas desde la cabeza y el resultado salía de menor a mayor; lo resolví empezando por la cola y retrocediendo con el puntero anterior. Otra dificultad fue manejar el final de una de las listas: hay que revisar null en cada puntero y copiar el resto de la otra lista.